KVFold: Expanding Cache Capacity for LLM Agents with Lossless Compression

Working draft
Author information to be supplied

***Abstract*—LLM agents often reuse long prompt prefixes when continuing tasks or resuming sessions. Caching the corresponding key/value (KV) states avoids repeated prefill, but capacity limits request concurrency and retained history. KVFold uses lossless exponent coding to fit more KV into GPU memory and external storage. Pool mode stores FragPack records on the GPU. Prefill decodes them into BF16 for FlashAttention-3; decode reconstructs operands within a fused attention kernel. Tier mode uses a separate record format in LMCache and leaves the GPU cache layout and attention implementation unchanged. In recorded AgentX runs with a fixed 54 GiB KV budget per GPU, pool mode increases usable capacity from 1.18M to 1.59M tokens. Visitors extend and edit a coding-agent prompt to inspect prefix reuse and KV reconstruction. They then explore recorded runs to see how added capacity affects concurrent requests and returning sessions.**

I. Introduction

Consider a coding agent investigating a timeout. Its prompt includes repository context, a bug report, and the investigation history. If the user then asks for a regression test, most of the prompt remains unchanged. Prefix caching avoids repeated prefill by reusing the keys and values already computed for this shared context. vLLM stores KV in paged blocks [1], while LMCache retains reusable prefixes in DRAM and NVMe across requests [2].

For Qwen3-30B-A3B, each cached token requires 96 KiB of BF16 KV across all layers. With many concurrent sessions, requests may wait for free GPU blocks even when external storage holds their prefixes. External tiers relieve GPU memory pressure by storing inactive histories, but can also fill up. If a resumed session's prefix has been evicted, the engine must recompute it. Capacity thus limits both concurrency and history reuse.

KVFold uses lossless compression to fit more KV into the available GPU memory and external storage. The codec preserves each BF16 value's low byte, assigns short codes to frequent high-byte values, and uses escapes for the remaining values. Unlike quantization methods such as KIVI [3] or lossy context codecs such as CacheGen [4], it reconstructs the original KV bits. The main *pool mode* stores completed GPU blocks as FragPack records. During prefill, KVFold gathers and decodes the context's KV into a BF16 workspace for FlashAttention-3 (FA3) [5]. During decode, it reconstructs operands within a fused attention kernel. An alternative *tier mode* stores compressed KV in LMCache while retaining the native GPU cache layout and attention implementation. Both modes share the coding principle and offline calibration, but use different formats for attention and external storage.

The demonstration follows the agent as it continues a task, runs alongside other sessions, and resumes an earlier investigation. Interactive requests show prefix reuse, external restoration, and KV byte checks. Recorded AgentX workloads [6] show the capacity gain at scale: with a 54 GiB KV budget per GPU, pool mode holds 1.59M tokens compared with 1.18M for the uncompressed baseline. With admission capped at 12 requests, mean engine time to first token (TTFT) falls from 5.16 to 1.94 s, excluding time spent waiting at the proxy.

II. System Description

KVFold integrates with vLLM 0.30 and the multiprocess cache server in LMCache 0.5.6 (Figure 1). vLLM schedules requests and allocates 128-token blocks. LMCache handles external lookup, LRU eviction, write-through, and prefetch.

A. Compressing closed blocks in the GPU pool

New KV enters a BF16 staging buffer as each block fills. At each layer, the backend writes the new values, computes attention, and encodes newly completed blocks. These records remain immutable in the layer's compressed pool. Incomplete blocks stay in staging so that later tokens can be appended.

A FragPack record stores one block's K or V for a layer and KV head: 128 tokens with 128 dimensions each, or 32 KiB in BF16. Seven frequent high-byte values have direct three-bit codes. Other values use a four-bit escape selector to choose among 15 additional values or fall back to a raw high byte. K and V use separate tables, calibrated for each layer and KV head. Low bytes are stored unchanged. Dense streams store scalars in the order required by the attention kernel's threads (Figure 2).

A block's K and V records share a page within each layer. For four KV heads, a page occupies 190,464 B instead of 262,144 B. The same GPU budget therefore holds 1.376 times as many pages before accounting for auxiliary backend buffers. Records that exceed the encoding limit or do not fit in a shared page are stored uncompressed in overflow slots.

B. Computing attention from compressed KV

Prefill with gather + FA3. When a request extends the context, the adapter gathers the context's KV into a BF16 workspace, decoding compressed records and copying values from open blocks. An unmodified FA3 kernel computes attention from this workspace, which holds the current layer's KV. Completed blocks remain compressed in each layer's pool.

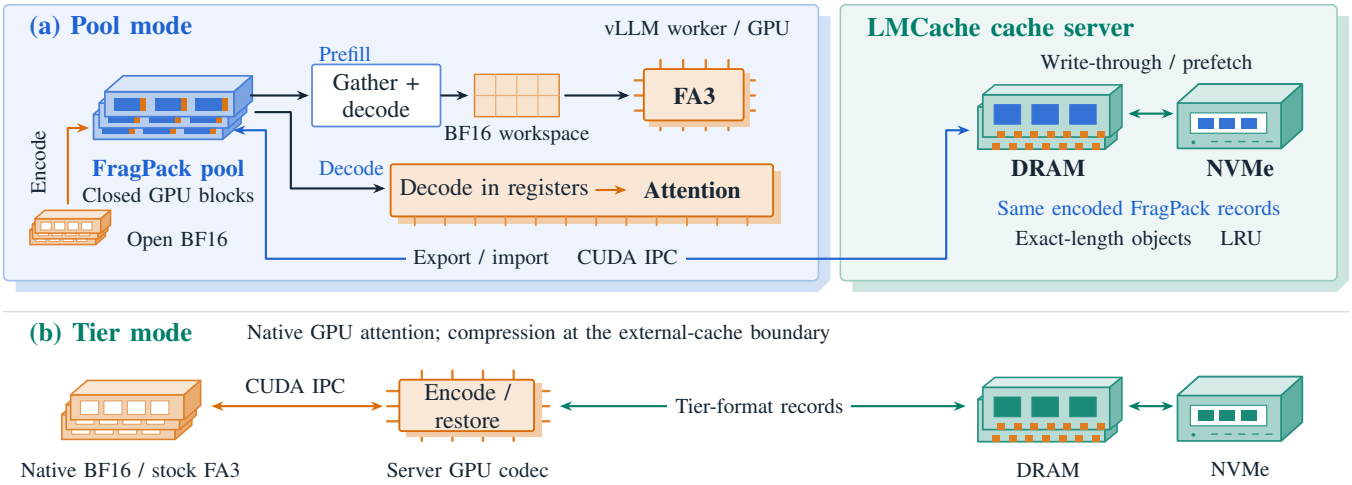


Fig. 1. KVFold's two integration paths. Pool mode stores FragPack records on the GPU and exports the same records to LMCache. Tier mode keeps the native GPU cache layout; the cache server encodes and restores KV. In both modes, LMCache manages storage in DRAM and NVMe, including eviction, write-through, and prefetch. Completed blocks are encoded; open blocks retain BF16 values for updates and attention.

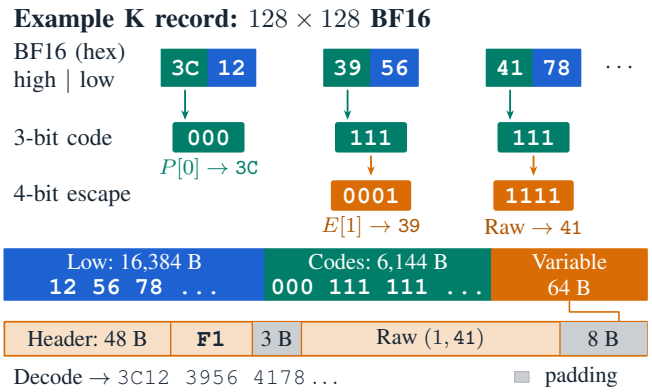


Fig. 2. Illustrative FragPack record (not to scale). The three scalars show direct lookup, an extended-table escape, and a raw escape. Each raw patch stores an escape's position in the escape sequence and its original high byte. The variable region uses 64 B in this example, within the format's 2,048 B limit. Recombining the high and low bytes recovers the original BF16 values exactly.

Decode with fused attention. For each generated token, a kernel divides the request's context into page ranges. It prefetches encoded records into shared memory and reconstructs operands in registers for tensor-core attention. A final kernel combines partial softmax states across the ranges. This path avoids writing the decoded context to GPU global memory and supports decode-only CUDA graph replay. A plugin loads the backend without changing vLLM's source.

Both paths reconstruct the original KV bits exactly. The console reports KV byte checks separately from generated text because fused attention and FA3 can use different reduction orders.

C. Storing KV in LMCache

Pool mode exports completed attention records from all layers into a relocatable blob containing dimensions, table identity, and lengths. LMCache stores each 128-token block at its actual length in DRAM and NVMe. On import, the server places records in GPU pages and updates their record tables. Failed validation triggers recomputation. The cache server accesses worker pools through CUDA IPC to run export and import kernels on the corresponding GPU, with one synchronization per load request.

Tier mode preserves the native GPU cache layout and attention implementation while compressing external KV. Each chunk contains one block's KV across all layers. Its per-layer, per-head K/V tables share the pool's calibration. They use only 14 symbols, seven primary and seven secondary, compared with FragPack's 22. Values are stored in token order. A three-bit primary code can escape to a second three-bit code and, if needed, a literal high byte. Low bytes remain unchanged. FragPack instead uses fragment order and four-bit escape selectors. Grouped indices locate escape streams for groups of eight 256-scalar tiles. Codebooks are fixed at startup and identified by a table digest. Chunks exceeding the raw storage limit remain uncompressed.

Each tensor-parallel (TP) rank stores exact-length objects, aligned to 4 KiB for NVMe direct I/O. Once the step producing the KV finishes, the server encodes full chunks from native pages into a bounded GPU pool. A background worker copies and commits the records to DRAM. On load, compressed records are decoded into destination pages. Raw records and partially shared leading chunks use a scratch buffer before scattering into pages. CUDA events order copies, decoding, and page reuse without host waits on the load path. LMCache uses stored lengths for NVMe prefetch, allowing the same budget to retain more history.

III. Demonstration

Visitors follow the timeout investigation through prefix reuse, GPU capacity limits, and session resumption (Figure 3). The console distinguishes live from recorded data and lists the model, TP degree, KV type, mode, and cache budgets. Modes

and codebooks are fixed at startup; selectors choose recorded configurations.

Setup. The interactive session runs Qwen3-4B-Instruct-2507 in eager pool mode on one H800 PCIe GPU. It uses BF16 weights and KV, TP 1, 128-token blocks, a 4 GiB GPU KV budget, and 1 GiB of LMCache DRAM.

The recorded capacity experiments used Qwen3-30B-A3B-Instruct with FP8 weights and BF16 KV, TP 2, and two H800 SXM GPUs connected over PCIe. vLLM used 128-token blocks and 8,192-token chunked prefill. AgentX harness 1.0.7 replayed `cc_traces_weka_062126_256k` using seed 42 and the recorded think times. P counts live root sessions; a FIFO proxy admits at most S concurrent requests. Measurements covered 1,800 seconds after priming and ten warmup requests. Requests without a first token by the end of the window were excluded. *Engine TTFT* excludes proxy wait. *Goodput* counts requests/minute with engine TTFT at most 2 s and inter-token latency (ITL) at most 40 ms. These measurements and output checks predate the shared calibration and grouped index described in Section II. The recorded tier runs used per-plane codebooks and the earlier tile index.

A. Step 1: extend a task and inspect prefix reuse

Visitors use *Build* to submit repository context, a bug report, and "Diagnose the timeout failure." They then use *Append* to add "Write a regression test." A prefix strip shows reused

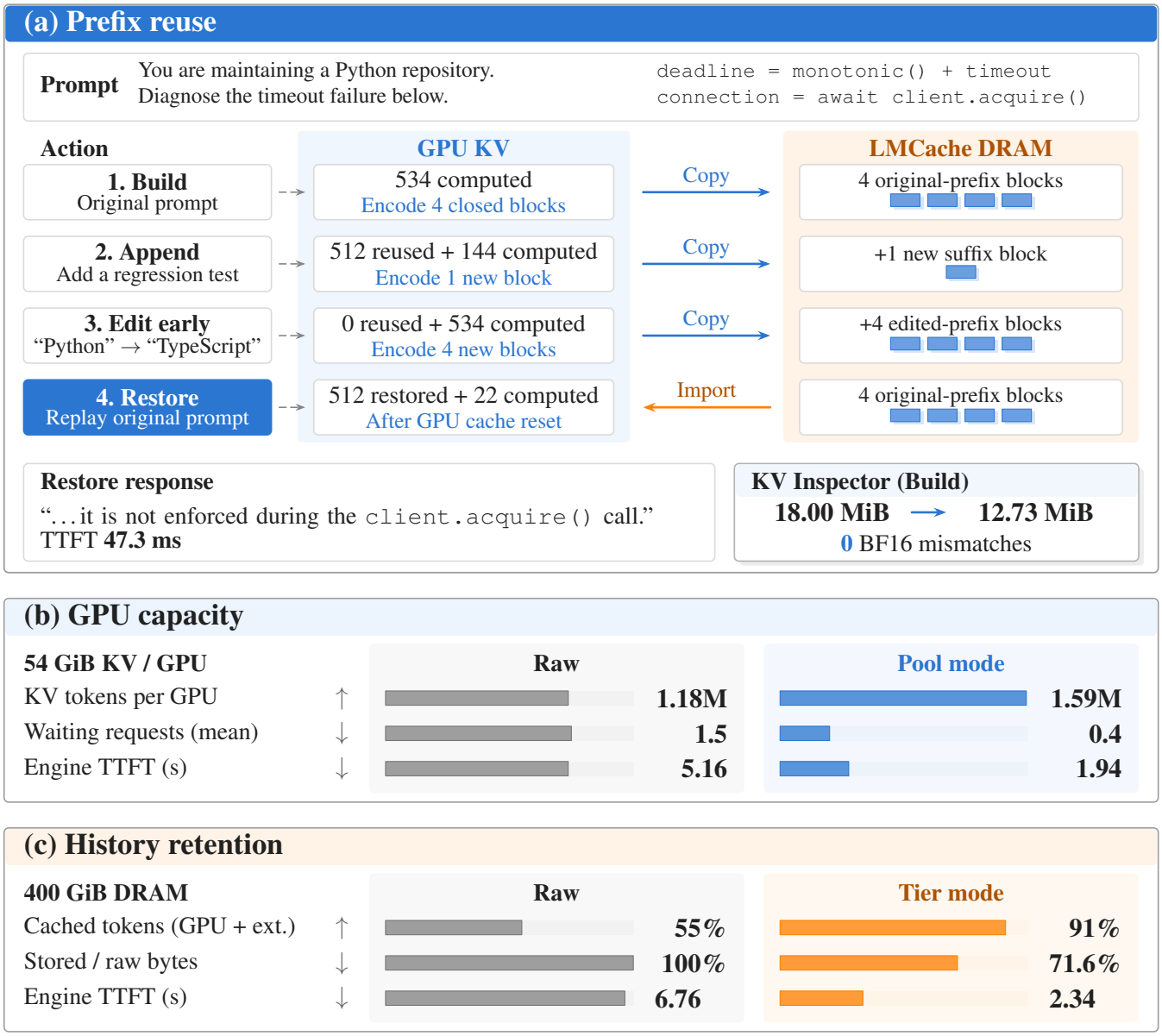


Fig. 3. Recomposed views. (a) Saved Qwen3-4B session. Dashed arrows mark requests; GPU–DRAM arrows transfer compressed blocks. Copy retains GPU records. Inspector: BF16/blob sizes and independent export/import after Build (576 records). (b,c) Archived Qwen3-30B AgentX: $P = 64$, 400 GiB DRAM; (b) $S = 12$, 2 TiB NVMe; (c) $S = 8$, 640 GiB NVMe. Paired bars share scales; cached tokens include GPU/external hits; raw bytes are normalized to 100%.

TABLE I
GPU CAPACITY AND PERFORMANCE IN THE RECORDED RUNS: 54 GIB KV PER GPU, $P = 64$, 400 GIB DRAM, AND 2 TIB NVME. WAIT DENOTES THE MEAN NUMBER OF REQUESTS WAITING IN VLLM; TTFT IS MEAN ENGINE TTFT IN SECONDS.

S	Mode	KV tokens	Wait	TTFT	Goodput
8	Raw	1.18M	0.3	1.69	15.1
8	Pool	1.59M	0.2	1.34	16.2
12	Raw	1.18M	1.5	5.16	10.6
12	Pool	1.59M	0.4	1.94	14.9

TABLE II
EXTERNAL CACHE CAPACITY AND PERFORMANCE IN THE RECORDED RUNS: $P = 64$, $S = 8$, AND 400 GIB DRAM. CACHED DENOTES THE SHARE OF PROMPT TOKENS SERVED FROM THE GPU OR EXTERNAL CACHE; TTFT IS MEAN ENGINE TTFT IN SECONDS.

NVMe	Mode	Cached	TTFT	Goodput
640 GiB	Raw	0.55	6.76	4.6
640 GiB	Tier	0.91	2.34	13.5
2 TiB	Raw	0.94	1.69	15.1
2 TiB	Tier	0.94	1.85	15.4

and new tokens alongside cached-token counts, TTFT, and the model's response. *Edit early* limits reuse to complete 128-token blocks before the changed token.

Restore waits for external saves, resets the GPU prefix cache, and replays the original prompt. The saved session restored 512 tokens from LMCache DRAM.

The inspector compares decoded KV from an already encoded block before and after export through host memory and import into an independent GPU scratch store. It displays sizes, checksums, and mismatch counts. All layers of the captured 128-token block matched byte for byte. Separate CPU samples exercise escapes and raw fallback.

A historical check found identical greedy outputs for 192 repeat-trace requests in tier and raw-tier modes with matching execution paths.

B. Step 2: explore GPU capacity limits

Visitors select $S = 8$ or $S = 12$ to compare raw and pool modes under equal GPU KV budgets (Table I). External tiers retain the working set, with a cached-token share of 0.94 in all runs. These runs examine GPU pressure when prefixes are cached.

Pool mode reserves 1.3 GiB within the KV budget for staging, overflow, and the prefill workspace. Cache-server CUDA contexts and transfer buffers are accounted for separately; codec buffers occupy 1.0 GiB per GPU outside vLLM's budget.

At $S = 12$, pool mode keeps TTFT close to raw mode at $S = 8$. At the higher cap, pool mode has fewer waiting requests and higher goodput than raw mode.

C. Step 3: revisit sessions under storage pressure

Visitors select 640 GiB or 2 TiB NVMe to compare retained history, stored bytes, TTFT, and goodput (Table II). Raw and tier modes share the native GPU KV layout, FA3, and LMCache's LRU, write-through, and prefetch policies. They allocate different record sizes.

At 640 GiB NVMe, tier mode's compressed-to-raw size ratio of 0.716 allows about 1.4 times as much KV to fit (Figure 3(c)). At 2 TiB, the raw working set fits and cached-token shares converge (Table II). Input throughput in tier mode is then 26.9 thousand tokens/s versus 27.5 for raw storage, a 2.2% reduction.

A separate replay of 48 prompts, with one request in flight, compares DRAM and NVMe hits. The prompts' KV occupies 261 GiB raw and 188 GiB compressed. At 220 GiB DRAM, compressed KV fits in memory, while some raw prefixes require NVMe reads. For 8K–128K prompts, mean warm TTFT with raw storage is 1.63–1.68 times that of tier mode. At 400 GiB DRAM, both representations fit and the gap narrows. The console displays these recorded residency and latency comparisons.

IV. CONCLUSION

KVFold fits more KV into GPU memory and external cache tiers. Visitors inspect prefix reuse, restore KV from LMCache, and check a model block before and after transfer. Recorded comparisons show how the additional capacity affects request queues and history reuse under fixed budgets.

ACKNOWLEDGMENT

REFERENCES

[1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. [Online]. Available: https://arxiv.org/abs/2309.06180

[2] Y. Liu, Y. Cheng, J. Yao, Y. An, X. Chen, S. Feng, Y. Huang, S. Shen, R. Zhang, K. Du, and J. Jiang, "LMCache: An efficient KV cache layer for enterprise-scale LLM inference," *arXiv preprint arXiv:2510.09665*, 2025. [Online]. Available: https://arxiv.org/abs/2510.09665

[3] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "KIVI: A tuning-free asymmetric 2bit quantization for KV cache," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 32 332–32 344. [Online]. Available: https://proceedings.mlr.press/v235/liu24bz.html

[4] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan, M. Maire, H. Hoffmann, A. Holtzman, and J. Jiang, "CacheGen: KV cache compression and streaming for fast large language model serving," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024. [Online]. Available: https://arxiv.org/abs/2310.07240

[5] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," *arXiv preprint arXiv:2407.08608*, 2024. [Online]. Available: https://arxiv.org/abs/2407.08608

[6] SemiAnalysis, "InferenceX AgentX: Methodology and datasets," https://inferencex.semianalysis.com/agentx, 2026.